project

Liyabona 2548620

2026-10-25

##Section A

```r
# 1. Initialization and Setup
set.seed(42) # For reproducible results
num_people <- 10000

# 2. Define Age Groups based on exact proportions
age_probs <- c(27.88, 30.5, 11.9) / 70.28
age_groups <- sample(
  c("18-35 years", "35-59 years", "Over 60"),
  size = num_people,
  replace = TRUE,
  prob = age_probs
)

# 3. Define Gender (Ratio of Men to Women is 1 : 1.1123452164)
ratio_women <- 1.1123452164
prob_men <- 1 / (1 + ratio_women)
prob_women <- ratio_women / (1 + ratio_women)

gender <- sample(
  c("Male", "Female"),
  size = num_people,
  replace = TRUE,
  prob = c(prob_men, prob_women)
)

# 4. Define Dwelling Type using Conditional Matrix Logic
dwelling_types <- character(num_people)

for (i in 1:num_people) {
  if (age_groups[i] == "Over 60") {
    dwelling_types[i] <- sample(
      c("Formal Dwelling", "Informal Settlement", "Traditional Dwelling",
        "Homeless"),
      size = 1, prob = c(0.150, 0.048, 0.800, 0.002)
    )
  } else if (age_groups[i] == "18-35 years") {
    dwelling_types[i] <- sample(
      c("Formal Dwelling", "Informal Settlement", "Traditional Dwelling",
        "Homeless"),
      size = 1, prob = c(0.248, 0.740, 0.010, 0.002)
    )
```

```r
  } else if (age_groups[i] == "35-59 years") {
    # Updated probabilities applied as requested
    dwelling_types[i] <- sample(
      c("Formal Dwelling", "Informal Settlement", "Traditional Dwelling",
        "Homeless"),
      size = 1, prob = c(0.588, 0.280, 0.130, 0.002)
    )
  }
}

# 5. Define Infrastructure Access Nested within Dwelling Type
electricity_cooking <- character(num_people)
water_access        <- character(num_people)
toilet_access       <- character(num_people)

for (i in 1:num_people) {
  current_dwelling <- dwelling_types[i]

  if (current_dwelling == "Formal Dwelling") {
    electricity_cooking[i] <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.95, 0.05))
    water_access[i]        <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.99, 0.01))
    toilet_access[i]       <- sample(c("Flushing Toilet", "No Access",
              "Non-Flushing Toilet"), 1, prob = c(0.98, 0.00, 0.02))

  } else if (current_dwelling == "Informal Settlement") {
    electricity_cooking[i] <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.40, 0.60))
    water_access[i]        <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.85, 0.15))
    toilet_access[i]       <- sample(c("Flushing Toilet", "No Access",
                       "Non-Flushing Toilet"), 1, prob = c(0.45, 0.55, 0.00))

  } else if (current_dwelling == "Traditional Dwelling") {
    electricity_cooking[i] <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.15, 0.85))
    water_access[i]        <- sample(c("Access", "No Access"), 1,
                                     prob = c(0.35, 0.65))
    toilet_access[i]       <- sample(c("Flushing Toilet", "No Access",
                                       "Non-Flushing Toilet"), 1,
                                     prob = c(0.05, 0.05, 0.90))

  } else if (current_dwelling == "Homeless") {
    electricity_cooking[i] <- "No Access"
    water_access[i]        <- "No Access"
    toilet_access[i]       <- "No Access"
  }
}

# 6. Combine into final R DataFrame
simulation_data <- data.frame(
  Person_ID = 1:num_people,
```

```
  Age_Group = age_groups,
  Gender = gender,
  Dwelling = dwelling_types,
  Electricity_Cooking = electricity_cooking,
  Water = water_access,
  Toilet = toilet_access,
  stringsAsFactors = FALSE
)


cat("1. VERIFICATION: Dwelling Type Percentages by Age Group (Row %):\n")
```

```
## 1. VERIFICATION: Dwelling Type Percentages by Age Group (Row %):
```

```
dwelling_cross_tab <- table(simulation_data$Age_Group, simulation_data$Dwelling)
print(round(prop.table(dwelling_cross_tab, margin = 1) * 100, 2))
```

```
##
##               Formal Dwelling Homeless Informal Settlement Traditional Dwelling
##   18-35 years           25.52     0.30               73.22                 0.96
##   35-59 years           58.94     0.09               27.80                13.16
##   Over 60               14.30     0.12                5.42                80.15
```

```
cat("\n2. Water Access Percentages within Dwelling Type:\n")
```

```
##
## 2. Water Access Percentages within Dwelling Type:
```

```
print(round(prop.table(table(simulation_data$Dwelling, simulation_data$Water),
                       margin = 1) * 100, 2))
```

```
##
##                        Access No Access
##   Formal Dwelling       99.08      0.92
##   Homeless               0.00    100.00
##   Informal Settlement   83.59     16.41
##   Traditional Dwelling  34.12     65.88
```

```
cat("\n3. Toilet Access Counts across all Dwelling Types:\n")
```

```
##
## 3. Toilet Access Counts across all Dwelling Types:
```

```
print(table(simulation_data$Dwelling, simulation_data$Toilet))
```

```
##
##                        Flushing Toilet No Access Non-Flushing Toilet
##   Formal Dwelling                 3714         0                  102
##   Homeless                           0        18                    0
##   Informal Settlement             1882      2329                    0
##   Traditional Dwelling              85        93                 1777
```

```
# 6. Create the structured flat table layout
synthetic_population <- data.frame(
  `Person ID` = paste("Person", 1:num_people),
  Gender = gender,
  `Age Group` = age_groups,
  `Dwelling Type` = dwelling_types,
```

```
  `Electricity for Cooking` = electricity_cooking,
  `Water Access` = water_access,
  `Toilet Access` = toilet_access,
  check.names = FALSE,
  stringsAsFactors = FALSE
)

# Export directly to a clean spreadsheet
write.csv(synthetic_population, "synthetic_population.csv", row.names = FALSE)
getwd()
```

```
## [1] "/Users/onako/Downloads"
```

```
# Step A: Calculate how many services each person has access to (out of 3)
services_count <-
  as.numeric(synthetic_population$`Electricity for Cooking` == "Access") +
  as.numeric(synthetic_population$`Water Access` == "Access") +
  as.numeric(synthetic_population$`Toilet Access` %in% c("Flushing Toilet",
                                                         "Non-Flushing Toilet"))

# Step B: Classify individuals into clean, intuitive Infrastructure Tiers
infra_tier <- character(num_people)
infra_tier[services_count == 3] <- "Full Access"
infra_tier[services_count == 0] <- "Zero Access"
infra_tier[services_count > 0 & services_count < 3] <- "Partial Access"

# Step C: Bind the Tiers and the unified Strata_ID back into your dataframe
synthetic_population$`Infrastructure Tier` <- infra_tier
synthetic_population$Strata_ID <- paste(synthetic_population$`Age Group`, "|", synthetic_population$`In

cat("\n--- FINAL STRATA DISTRIBUTION (Age Group x Infrastructure Tier) ---\n")
```

```
##
## --- FINAL STRATA DISTRIBUTION (Age Group x Infrastructure Tier) ---
```

```
strata_table <- table(synthetic_population$Strata_ID)
print(strata_table)
```

```
##
##    18-35 years | Full Access 18-35 years | Partial Access
##                          1376                          2422
##    18-35 years | Zero Access    35-59 years | Full Access
##                           179                          2609
## 35-59 years | Partial Access    35-59 years | Zero Access
##                          1644                            92
##        Over 60 | Full Access     Over 60 | Partial Access
##                           299                          1342
##        Over 60 | Zero Access
##                            37
```

```
strata_lists <- split(synthetic_population, synthetic_population$Strata_ID)
```

Proportional sampling

```
# 1. Define your total sample size
total_sample_size <- 1333
```

```r
# 2. Create a data frame of the strata counts from your existing output
strata_df <- data.frame(table(synthetic_population$Strata_ID))
colnames(strata_df) <- c("Strata_ID", "Population_Ni")

# 3. Calculate total population dynamically
total_population <- sum(strata_df$Population_Ni)

# 4. Apply proportional allocation formula
strata_df$Percentage <- strata_df$Population_Ni / total_population
strata_df$Raw_Sample <- strata_df$Percentage * total_sample_size

# 5. Round to whole numbers (using round() or a standard rounding function)
strata_df$Allocated_Sample_ni <- round(strata_df$Raw_Sample)

# 6. Verify total matches target sample size
cat("Target Sample Size:", total_sample_size, "\n")
```

```
## Target Sample Size: 1333
```

```r
cat("Allocated Sample Total:", sum(strata_df$Allocated_Sample_ni), "\n\n")
```

```
## Allocated Sample Total: 1333
```

```r
# View the final allocation table
print(strata_df[, c("Strata_ID", "Population_Ni", "Allocated_Sample_ni")])
```

```
##                      Strata_ID Population_Ni Allocated_Sample_ni
## 1    18-35 years | Full Access          1376                 183
## 2 18-35 years | Partial Access          2422                 323
## 3    18-35 years | Zero Access           179                  24
## 4    35-59 years | Full Access          2609                 348
## 5 35-59 years | Partial Access          1644                 219
## 6    35-59 years | Zero Access            92                  12
## 7        Over 60 | Full Access           299                  40
## 8     Over 60 | Partial Access          1342                 179
## 9        Over 60 | Zero Access            37                   5
```

#section c

```r
anticipate_nonresponse<- 0.03
adjusted_sample_size<- total_sample_size/(1-anticipate_nonresponse)
round(adjusted_sample_size)
```

```
## [1] 1374
```

```r
total_sample_size_adjusted <- 1374

# 2. Create a data frame of the strata counts and rename to adjusted_strata_df
adjusted_strata_df <- data.frame(table(synthetic_population$Strata_ID))
colnames(adjusted_strata_df) <- c("Strata_ID", "Population_Ni")

# 3. Calculate total population dynamically
total_population <- sum(adjusted_strata_df$Population_Ni)

# 4. Apply proportional allocation formula
adjusted_strata_df$Percentage <- adjusted_strata_df$Population_Ni / total_population
adjusted_strata_df$Raw_Adjusted_Sample <- adjusted_strata_df$Percentage * total_sample_size_adjusted
```

```r
# 5. Round to whole numbers using standard rounding
adjusted_strata_df$Adjusted_Sample_ni <- round(adjusted_strata_df$Raw_Adjusted_Sample)

# 6. Verify total matches target sample size
cat("Adjusted Target Sample Size:", total_sample_size_adjusted, "\n")
```

```
## Adjusted Target Sample Size: 1374
```

```r
cat("Adjusted Allocated Sample Total:", sum(adjusted_strata_df$Adjusted_Sample_ni), "\n\n")
```

```
## Adjusted Allocated Sample Total: 1374
```

```r
# View the final adjusted allocation table
print(adjusted_strata_df[, c("Strata_ID", "Population_Ni", "Adjusted_Sample_ni")])
```

```
##                      Strata_ID Population_Ni Adjusted_Sample_ni
## 1    18-35 years | Full Access          1376                189
## 2 18-35 years | Partial Access          2422                333
## 3    18-35 years | Zero Access           179                 25
## 4    35-59 years | Full Access          2609                358
## 5 35-59 years | Partial Access          1644                226
## 6    35-59 years | Zero Access            92                 13
## 7        Over 60 | Full Access           299                 41
## 8     Over 60 | Partial Access          1342                184
## 9        Over 60 | Zero Access            37                  5
```

```r
#SAMPLING WITHOUT REPLACEMENT
# Ensure reproducibility
set.seed(42)


strata_allocations <- c(
 "18-35 years | Full Access" = 189,
 "18-35 years | Partial Access" = 333,
 "18-35 years | Zero Access" = 25,
 "35-59 years | Full Access" = 358,
 "35-59 years | Partial Access" = 226,
 "35-59 years | Zero Access" = 13,
 "Over 60 | Full Access" = 41,
 "Over 60 | Partial Access" = 184,
 "Over 60 | Zero Access" = 5
)

# 2. Draw the sample using Simple Random Sampling Without Replacement (SRSWOR) per stratum
sample_list <- lapply(names(strata_allocations), function(stratum) {
 # Subset population to the current stratum
 stratum_pop <- subset(synthetic_population, Strata_ID == stratum)
 n_h <- strata_allocations[stratum]

 # Sample row indices without replacement
 sampled_indices <- sample(seq_len(nrow(stratum_pop)), size = n_h, replace = FALSE)

 return(stratum_pop[sampled_indices, ])
})
```

```r
# 3. Combine into a unified sample data frame
selected_sample <- do.call(rbind, sample_list)

# 4. Add the first-phase base sampling weight (d_i = N_h / n_h)
# Calculate population stratum counts dynamically
N_h_counts <- table(synthetic_population$Strata_ID)

selected_sample$base_weight <- sapply(selected_sample$Strata_ID, function(id) {
 as.numeric(N_h_counts[id]) / strata_allocations[id]
})
```

Non response and Outcome variable

```r
set.seed(42) # Ensure strict reproducibility across steps
phase1_sample <- data.frame()
for (stratum in names(strata_allocations)) {
 n_ch <- strata_allocations[stratum]
 pool <- subset(synthetic_population, Strata_ID == stratum)
 sampled_units <- pool[sample(1:nrow(pool), n_ch, replace = FALSE), ]
 phase1_sample <- rbind(phase1_sample, sampled_units)
}

# Define Phase 1 inclusion probabilities and design weights
pop_strata_counts <- table(synthetic_population$Strata_ID)
phase1_sample$N_h <- as.numeric(pop_strata_counts[phase1_sample$Strata_ID])
phase1_sample$n_ch <- strata_allocations[phase1_sample$Strata_ID]

# Inclusion Probability Phase 1 (pi_i)
phase1_sample$Inclusion_Prob_Phase1 <- phase1_sample$n_ch / phase1_sample$N_h
# Direct Design Weight (d_i = 1 / pi_i)
phase1_sample$d_i <- 1 / phase1_sample$Inclusion_Prob_Phase1



log_odds_resp <- with(phase1_sample,
 0.5 +
 1.1 * as.numeric(`Age Group` == "Over 60") +
 0.4 * as.numeric(`Age Group` == "35-59 years") +
 0.8 * as.numeric(`Infrastructure Tier` == "Full Access") -
 0.6 * as.numeric(`Infrastructure Tier` == "Zero Access") -
 1.2 * as.numeric(`Dwelling Type` == "Homeless")
)
phase1_sample$True_Theta <- 1 / (1 + exp(-log_odds_resp))

# 3.2 Realize Phase 2 Sampling Process via Bernoulli trials (1 = Respondent, 0 = Non-Respondent)
phase1_sample$Respondent <- rbinom(nrow(phase1_sample), 1, phase1_sample$True_Theta)

# 3.3 Fit the explicit parametric Logit model using available auxiliary variables
logit_response_model <- glm(
 Respondent ~ `Age Group` + `Infrastructure Tier` + `Dwelling Type`,
 data = phase1_sample,
 family = binomial(link = "logit")
)
```

```r
# 3.4 Extract the Estimated Propensities (theta_hat_i)
phase1_sample$Estimated_Theta <- predict(logit_response_model, type = "response")



phase1_sample$gamma_i <- ifelse(phase1_sample$Respondent == 1, 1 / phase1_sample$Estimated_Theta, 0)

# Final Calibrated Sampling Weight (w_i = d_i * gamma_i)
phase1_sample$Final_Sampling_Weight <- phase1_sample$d_i * phase1_sample$gamma_i



log_odds_vote <- with(phase1_sample,
 -0.2 +
 1.3 * as.numeric(`Age Group` == "Over 60") +
 0.6 * as.numeric(`Age Group` == "35-59 years") +
 0.7 * as.numeric(`Infrastructure Tier` == "Full Access") -
 0.9 * as.numeric(`Infrastructure Tier` == "Zero Access") +
 0.5 * as.numeric(`Dwelling Type` == "Traditional Dwelling") -
 1.5 * as.numeric(`Dwelling Type` == "Homeless")
)
phase1_sample$Voting_Probability <- 1 / (1 + exp(-log_odds_vote))

# 5.2 Compute actual realized Bernoulli behavior outcomes
phase1_sample$Realized_Behavior <- rbinom(nrow(phase1_sample), 1, phase1_sample$Voting_Probability)

# 5.3 Enforce Two-Phase constraints: Outcome variable is ONLY observable for respondents
phase1_sample$Observed_Vote_Outcome <- ifelse(
 phase1_sample$Respondent == 1,
 phase1_sample$Realized_Behavior,
 NA # Missing/Unobserved for total non-response units
)



survey_microdata_export <- data.frame(
 # Unique Identifiers
 Unique_Unit_ID          = paste0("SU-", sprintf("%05d", phase1_sample$Person_ID)),

 # Stratum Identifiers
 Stratum_Identifier       = phase1_sample$Strata_ID,

 # Probability Metrics
 Phase1_Inclusion_Prob    = phase1_sample$Inclusion_Prob_Phase1,
 Estimated_Response_Prob  = phase1_sample$Estimated_Theta,
 Total_Inclusion_Prob     = phase1_sample$Inclusion_Prob_Phase1 * phase1_sample$Estimated_Theta,

 # Weighting Structures
 Design_Weight_di         = phase1_sample$d_i,
 Correction_Factor_gamma  = phase1_sample$gamma_i,
 Final_Sampling_Weight    = phase1_sample$Final_Sampling_Weight,

 # Outcomes & Indicators
```

```
 Survey_Response_Status   = ifelse(phase1_sample$Respondent == 1, "Respondent", "Non-Respondent"),
 Vote_Outcome             = phase1_sample$Observed_Vote_Outcome,

 # Relevant Auxiliary Variables (Known for both respondents and non-respondents)
 Auxiliary_Age_Group      = phase1_sample$`Age Group`,
 Auxiliary_Gender         = phase1_sample$Gender,
 Auxiliary_Dwelling_Type  = phase1_sample$`Dwelling Type`,
 Auxiliary_Infra_Tier     = phase1_sample$`Infrastructure Tier`,

 stringsAsFactors = FALSE,
 check.names = FALSE
)

# Export directly to spreadsheet
write.csv(survey_microdata_export, "survey_microdata_export.csv", row.names = FALSE)

# Execution Verification Summary
cat("=== SURVEY MICRODATA PROCESSING COMPLETION ===\n")
```

```
## === SURVEY MICRODATA PROCESSING COMPLETION ===
```

```
cat("Total Sampled Units in Phase 1 (n_c):", nrow(survey_microdata_export), "\n")
```

```
## Total Sampled Units in Phase 1 (n_c): 1374
```

```
cat("Total Phase 2 Respondents Realized (s_R):", sum(survey_microdata_export$Survey_Response_Status ==
```

```
## Total Phase 2 Respondents Realized (s_R): 1019
```

```
cat("Output File Saved Successfully: 'survey_microdata_export.csv'\n")
```

```
## Output File Saved Successfully: 'survey_microdata_export.csv'
```

Section D

```
library(survey)
```

```
## Warning: package 'survey' was built under R version 4.3.3
```

```
## Loading required package: grid
```

```
## Loading required package: Matrix
```

```
## Loading required package: survival
```

```
##
## Attaching package: 'survey'
```

```
## The following object is masked from 'package:graphics':
##
##     dotchart
```

```
# 1. Filter the microdata to keep only the completed responses (s_R = 1019 units)
respondents_data <- subset(survey_microdata_export, Survey_Response_Status == "Respondent")

# 2. Extract population stratum counts (N_h) dynamically from your 10,000 population frame
pop_strata_counts <- table(synthetic_population$Strata_ID)

# 3. Map the true population stratum sizes back onto our respondent dataset for the FPC
respondents_data$N_h <- as.numeric(pop_strata_counts[respondents_data$Stratum_Identifier])
```

```r
# 4. Correctly define the complex, two-phase calibrated survey design object
survey_design <- svydesign(
  ids     = ~1,                          # No clusters; elements sampled directly
  strata  = ~Stratum_Identifier,         # Stratification variable (Age x Infra Tier)
  weights = ~Final_Sampling_Weight,      # Calibrated weights (d_i * gamma_i)
  fpc     = ~N_h,                        # Stratum population sizes for FPC correction
  data    = respondents_data             # The respondent-only microdata matrix
)

# 5. Verify the design structure matches your sample layout
summary(survey_design)
```

```
## Stratified Independent Sampling design
## svydesign(ids = ~1, strata = ~Stratum_Identifier, weights = ~Final_Sampling_Weight,
##     fpc = ~N_h, data = respondents_data)
## Probabilities:
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.05313 0.10071 0.10558 0.10359 0.11564 0.11961
## Stratum Sizes:
##            18-35 years | Full Access 18-35 years | Partial Access
## obs                                139                          212
## design.PSU                         139                          212
## actual.PSU                         139                          212
##            18-35 years | Zero Access 35-59 years | Full Access
## obs                                 12                       310
## design.PSU                          12                       310
## actual.PSU                          12                       310
##            35-59 years | Partial Access 35-59 years | Zero Access
## obs                                  161                          5
## design.PSU                           161                          5
## actual.PSU                           161                          5
##            Over 60 | Full Access Over 60 | Partial Access Over 60 | Zero Access
## obs                           33                      145                      2
## design.PSU                    33                      145                      2
## actual.PSU                    33                      145                      2
## Population stratum sizes (PSUs):
##    18-35 years | Full Access 18-35 years | Partial Access
##                         1376                         2422
##    18-35 years | Zero Access    35-59 years | Full Access
##                          179                         2609
## 35-59 years | Partial Access    35-59 years | Zero Access
##                         1644                           92
##        Over 60 | Full Access     Over 60 | Partial Access
##                          299                         1342
##        Over 60 | Zero Access
##                           37
## Data variables:
##  [1] "Unique_Unit_ID"          "Stratum_Identifier"
##  [3] "Phase1_Inclusion_Prob"   "Estimated_Response_Prob"
##  [5] "Total_Inclusion_Prob"    "Design_Weight_di"
##  [7] "Correction_Factor_gamma" "Final_Sampling_Weight"
##  [9] "Survey_Response_Status"  "Vote_Outcome"
## [11] "Auxiliary_Age_Group"     "Auxiliary_Gender"
```

```
## [13] "Auxiliary_Dwelling_Type" "Auxiliary_Infra_Tier"
## [15] "N_h"
```

```
# We calculate the population mean proportion for Vote_Outcome.
# We set deff = TRUE to compute the design effect required in Question 3.
vote_estimation <- svymean(~Vote_Outcome, design = survey_design, deff = TRUE)



# Extracting each specific property from our estimation model object:
point_estimate     <- coef(vote_estimation)[]
estimated_variance <- vcov(vote_estimation)[1, 1]
standard_error     <- SE(vote_estimation)[]
conf_interval      <- confint(vote_estimation, level = 0.95)
coeff_of_variation <- cv(vote_estimation)[]
design_effect      <- deff(vote_estimation)

# Print a formatted summary directly to your console/markdown file
cat("=== SECTION D: SURVEY ESTIMATION RESULTS ===\n")
```

```
## === SECTION D: SURVEY ESTIMATION RESULTS ===
```

```
cat("Point Estimate (Weighted Voting Proportion): ", round(point_estimate, 4), "\n")
```

```
## Point Estimate (Weighted Voting Proportion):  0.6323
```

```
cat("Estimated Variance:                          ", round(estimated_variance, 6), "\n")
```

```
## Estimated Variance:                           0.000191
```

```
cat("Standard Error (SE):                         ", round(standard_error, 4), "\n")
```

```
## Standard Error (SE):                          0.0138
```

```
cat("95% Confidence Interval:                     [", round(conf_interval[1], 4), ",", round(conf_inter
```

```
## 95% Confidence Interval:                     [ 0.6052 , 0.6595 ]
```

```
cat("Coefficient of Variation (CV):                ", round(coeff_of_variation, 4), "\n")
```

```
## Coefficient of Variation (CV):                 0.0219
```

```
cat("Design Effect (Deff):                        ", round(design_effect, 4), "\n")
```

```
## Design Effect (Deff):                         0.9331
```

```
cat("============================================\n")
```

```
## ============================================
```

```
cat("=== QUESTION 4: SAMPLING WEIGHTS & INCLUSION PROBABILITIES ===\n\n")
```

```
## === QUESTION 4: SAMPLING WEIGHTS & INCLUSION PROBABILITIES ===
```

```
# 1. Extract and inspect a sample of the structural metrics from your respondents
weight_inspection <- respondents_data[, c("Unique_Unit_ID", "Stratum_Identifier",
                                          "Phase1_Inclusion_Prob", "Estimated_Response_Prob",
                                          "Total_Inclusion_Prob", "Final_Sampling_Weight")]

cat("--- First 5 Microdata Records For Respondents ---\n")
```

```
## --- First 5 Microdata Records For Respondents ---
```

```r
print(head(weight_inspection, 5))
```

```
##   Unique_Unit_ID        Stratum_Identifier Phase1_Inclusion_Prob
## 2            SU- 18-35 years | Full Access             0.1373547
## 3            SU- 18-35 years | Full Access             0.1373547
## 4            SU- 18-35 years | Full Access             0.1373547
## 5            SU- 18-35 years | Full Access             0.1373547
## 6            SU- 18-35 years | Full Access             0.1373547
##   Estimated_Response_Prob Total_Inclusion_Prob Final_Sampling_Weight
## 2               0.7660955            0.1052268              9.503284
## 3               0.7660955            0.1052268              9.503284
## 4               0.7660955            0.1052268              9.503284
## 5               0.7660955            0.1052268              9.503284
## 6               0.7686921            0.1055834              9.471183
```

```r
cat("\n")
```

```r
# 2. Compute summary diagnostics to explain the weight distribution
cat("--- Distribution Summary of Final Calibrated Sampling Weights ---\n")
```

```
## --- Distribution Summary of Final Calibrated Sampling Weights ---
```

```r
print(summary(respondents_data$Final_Sampling_Weight))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   8.361   8.647   9.471   9.826   9.930  18.823
```

##Section D SRSWOR

```r
# ======================================================================
# SINGLE REPLICATION: BENCHMARK SRSWOR (100% RESPONSE)
# ======================================================================
library(survey)

# 1. Enforce strict reproducibility for this single run
set.seed(42)

# 2. Define finite population size metrics
N_population <- nrow(synthetic_population)
n_sample_srs <- 1376

# 3. Generate individual True Vote Outcomes using your exact logit equation
log_odds_vote <- with(synthetic_population,
  -0.2 +
  1.3 * as.numeric(`Age Group` == "Over 60") +
  0.6 * as.numeric(`Age Group` == "35-59 years") +
  0.7 * as.numeric(`Infrastructure Tier` == "Full Access") -
  0.9 * as.numeric(`Infrastructure Tier` == "Zero Access") +
  0.5 * as.numeric(`Dwelling Type` == "Traditional Dwelling") -
  1.5 * as.numeric(`Dwelling Type` == "Homeless")
)
synthetic_population$True_Vote_Outcome <- rbinom(N_population, 1, 1 / (1 + exp(-log_odds_vote)))

# 4. Blindly sample element rows without replacement (No Stratification)
sampled_rows_srs <- sample(seq_len(N_population), size = n_sample_srs, replace = FALSE)
single_sim_srs   <- synthetic_population[sampled_rows_srs, ]
```

```r
# 5. Assume 100% response across the drawn cohort
single_sim_srs$Observed_Vote_SRS <- single_sim_srs$True_Vote_Outcome

# 6. Apply equal expansion inclusion properties and Finite Population Correction
single_sim_srs$Sampling_Weight_srs <- 1 / (n_sample_srs / N_population)
single_sim_srs$Pop_Total_FPC_srs   <- N_population

# 7. Configure unstratified survey design object
single_design_srs <- svydesign(
  ids     = ~1,
  strata  = NULL,
  weights = ~Sampling_Weight_srs,
  fpc     = ~Pop_Total_FPC_srs,
  data    = single_sim_srs
)

# 8. Compute design-based parameters
single_estimate <- svymean(~Observed_Vote_SRS, design = single_design_srs)

# ======================================================================
# EXTRACTION
# ======================================================================
point_estimate_val <- as.numeric(coef(single_estimate))
estimated_var_val  <- as.numeric(vcov(single_estimate))

# Print clean, scannable summary diagnostics directly to the console
cat("=== SINGLE SRSWOR SIMULATION METRICS ===\n")
```

```
## === SINGLE SRSWOR SIMULATION METRICS ===
```

```r
cat("Point Estimate (Sample Mean Proportion): ", round(point_estimate_val, 6), "\n")
```

```
## Point Estimate (Sample Mean Proportion):  0.672238
```

```r
cat("Estimated Variance:                      ", sprintf("%.8f", estimated_var_val), "\n")
```

```
## Estimated Variance:                      0.00013819
```

```r
cat("========================================\n")
```

```
## ========================================
```

##Section E :SRSWOR

```r
# ======================================================================
# ESTABLISH FIXED POPULATION PARAMETERS (GROUND TRUTH)
# ======================================================================
set.seed(42) # Ensure strict reproducible population behaviors

# 1. Apply your precise logistic voting equation to the ENTIRE city population
log_odds_vote_population <- with(synthetic_population,
  -0.2 +
  1.3 * as.numeric(`Age Group` == "Over 60") +
  0.6 *  as.numeric(`Age Group` == "35-59 years") +
  0.7 * as.numeric(`Infrastructure Tier` == "Full Access") -
  0.9 * as.numeric(`Infrastructure Tier` == "Zero Access") +
  0.5 * as.numeric(`Dwelling Type` == "Traditional Dwelling") -
```

```
  1.5 * as.numeric(`Dwelling Type` == "Homeless")
)

# 2. Convert log-odds to actual individual response probabilities
synthetic_population$True_Voting_Prob <- 1 / (1 + exp(-log_odds_vote_population))

# 3. Realize the latent voting behavior permanently across the frame
synthetic_population$True_Vote_Outcome <- rbinom(
  n = nrow(synthetic_population),
  size = 1,
  prob = synthetic_population$True_Voting_Prob
)

#This is the globa truth under the logistic regression model not the true turnout (0.612)
true_population_proportion <- mean(synthetic_population$True_Vote_Outcome)

cat("=== GLOBAL GROUND TRUTH ESTABLISHED ===\n")
```

```
## === GLOBAL GROUND TRUTH ESTABLISHED ===
```

```
cat("True Population Voting Proportion (P):", round(true_population_proportion, 4), "\n\n")
```

```
## True Population Voting Proportion (P): 0.6734
```

```
# ======================================================================
# MONTE CARLO SIMULATION FOR SRSWOR BENCHMARK (100% RESPONSE)
# ======================================================================
library(survey)

B <- 500
n_sample_srs <- 1376
N_population <- nrow(synthetic_population)

# Pre-allocate storage dataframe for tracking metrics across 500 loops
srswor_sim_results <- data.frame(
  Simulation_ID      = integer(B),
  Sample_Size        = integer(B),
  Point_Estimate     = numeric(B),
  Estimated_Variance = numeric(B),
  Standard_Error     = numeric(B),
  CI_Lower           = numeric(B),
  CI_Upper           = numeric(B),
  CV                 = numeric(B),
  Design_Effect      = numeric(B),
  stringsAsFactors   = FALSE
)

for (b in seq_len(B)) {

  # 1. Blindly extract row indices from the population frame (No Stratification)
  sampled_rows_srs <- sample(seq_len(N_population), size = n_sample_srs, replace = FALSE)
  selected_sample_srs <- synthetic_population[sampled_rows_srs, ]

  # 2. ENFORCE 100% RESPONSE: Directly read the fixed truth (No dropouts)
  selected_sample_srs$Observed_Vote_SRS <- selected_sample_srs$True_Vote_Outcome
```

```r
  # 3. Calculate identical base inclusion probabilities and expansion weights
  selected_sample_srs$Inclusion_Prob_srs <- n_sample_srs / N_population
  selected_sample_srs$Sampling_Weight_srs <- 1 / selected_sample_srs$Inclusion_Prob_srs
  selected_sample_srs$Pop_Total_FPC_srs <- N_population

  # 4. Configure unstratified survey design structure
  survey_design_srs <- svydesign(
    ids     = ~1,
    strata  = NULL,
    weights = ~Sampling_Weight_srs,
    fpc     = ~Pop_Total_FPC_srs,
    data    = selected_sample_srs
  )

  # 5. Estimate design-based population proportion mean
  vote_est_srs <- svymean(~Observed_Vote_SRS, design = survey_design_srs, deff = TRUE)

  # 6. Extract evaluation metrics via survey package methods
  ci_srs <- confint(vote_est_srs, level = 0.95)

  srswor_sim_results[b, ] <- list(
    Simulation_ID      = b,
    Sample_Size        = n_sample_srs,
    Point_Estimate     = as.numeric(coef(vote_est_srs)[1]),
    Estimated_Variance = as.numeric(vcov(vote_est_srs)[1, 1]),
    Standard_Error     = as.numeric(SE(vote_est_srs)[1]),
    CI_Lower           = as.numeric(ci_srs[1, 1]),
    CI_Upper           = as.numeric(ci_srs[1, 2]),
    CV                 = as.numeric(cv(vote_est_srs)[1]),
    Design_Effect      = as.numeric(deff(vote_est_srs)[1])
  )
}

# Export full simulation matrix to clean .csv output
write.csv(srswor_sim_results, "SRSWOR_Shared_Truth_Simulation_Results.csv", row.names = FALSE)

cat("=== BENCHMARK SRSWOR SIMULATION COMPLETE ===\n")
```

```
## === BENCHMARK SRSWOR SIMULATION COMPLETE ===
```

```r
cat("Results saved successfully to 'SRSWOR_Shared_Truth_Simulation_Results.csv'\n")
```

```
## Results saved successfully to 'SRSWOR_Shared_Truth_Simulation_Results.csv'
```

```r
# ======================================================================
# CALCULATE ACCURACY & PERFORMANCE MEASURES
# ======================================================================
estimates_srs    <- srswor_sim_results$Point_Estimate
errors_srs       <- estimates_srs - true_population_proportion
lower_bounds_srs <- srswor_sim_results$CI_Lower
upper_bounds_srs <- srswor_sim_results$CI_Upper

# Compute summary performance criteria
empirical_bias_srs <- mean(errors_srs)
relative_bias_srs  <- (empirical_bias_srs / true_population_proportion) * 100
```

```r
empirical_se_srs   <- sd(estimates_srs)
avg_estimated_se   <- mean(srswor_sim_results$Standard_Error)
mse_srs            <- mean(errors_srs^2)
rmse_srs           <- sqrt(mse_srs)

# Empirical coverage performance
coverage_vector_srs <- (lower_bounds_srs <= true_population_proportion) &
                       (upper_bounds_srs >= true_population_proportion)
coverage_pct_srs    <- mean(coverage_vector_srs) * 100
mc_se_coverage_srs  <- sqrt((mean(coverage_vector_srs) * (1 - mean(coverage_vector_srs))) / B) * 100
avg_ci_width_srs    <- mean(upper_bounds_srs - lower_bounds_srs)

cat("\n=== MONTE CARLO EVALUATION: SRSWOR BENCHMARK ===\n")
```

```
##
## === MONTE CARLO EVALUATION: SRSWOR BENCHMARK ===
```

```r
cat("Empirical Bias:                  ", round(empirical_bias_srs, 6), "\n")
```

```
## Empirical Bias:                  -0.000375
```

```r
cat("Relative Bias:                   ", round(relative_bias_srs, 2), "%\n")
```

```
## Relative Bias:                   -0.06 %
```

```r
cat("Empirical Standard Error:        ", round(empirical_se_srs, 4), "\n")
```

```
## Empirical Standard Error:        0.0124
```

```r
cat("Average Analytical SE:           ", round(avg_estimated_se, 4), "\n")
```

```
## Average Analytical SE:           0.0117
```

```r
cat("Root Mean Squared Error (RMSE):  ", round(rmse_srs, 4), "\n")
```

```
## Root Mean Squared Error (RMSE):  0.0124
```

```r
cat("95% Confidence Interval Coverage: ", round(coverage_pct_srs, 2), "%\n")
```

```
## 95% Confidence Interval Coverage:  94.4 %
```

```r
cat("Monte Carlo SE of Coverage (pp):  ", round(mc_se_coverage_srs, 4), " percentage points\n")
```

```
## Monte Carlo SE of Coverage (pp):   1.0282  percentage points
```

```r
cat("Average Confidence Interval Width:", round(avg_ci_width_srs, 4), "\n")
```

```
## Average Confidence Interval Width: 0.046
```

```r
cat("Average Design Effect (Deff):     ", round(mean(srswor_sim_results$Design_Effect), 4), "\n")
```

```
## Average Design Effect (Deff):      1
```

##Section E

```r
# 1. Set the number of simulations
B <- 500

# Ensure reproducibility
set.seed(42)
```

```r
strata_allocations <- c(
  "18-35 years | Full Access" = 189,
  "18-35 years | Partial Access" = 333,
  "18-35 years | Zero Access" = 25,
  "35-59 years | Full Access" = 358,
  "35-59 years | Partial Access" = 226,
  "35-59 years | Zero Access" = 13,
  "Over 60 | Full Access" = 41,
  "Over 60 | Partial Access" = 184,
  "Over 60 | Zero Access" = 5
)

# Verify the total allocated sample size
stopifnot(sum(strata_allocations) == 1374)

# Calculate population counts for each stratum
pop_strata_counts <- table(synthetic_population$Strata_ID)

# Check that every requested stratum exists
stopifnot(all(names(strata_allocations) %in%
                names(pop_strata_counts)))

# Check that no stratum allocation exceeds its population
stopifnot(all(
  strata_allocations <=
    as.numeric(pop_strata_counts[names(strata_allocations)])
))



# Store the individual-level data from every simulation
simulation_data <- vector("list", B)

# Store summary statistics from every simulation
simulation_summary <- vector("list", B)



for (b in seq_len(B)) {


  sample_list <- lapply(names(strata_allocations), function(stratum) {

    # Select the population belonging to the current stratum
    stratum_pop <- synthetic_population[
      synthetic_population$Strata_ID == stratum, ,
      drop = FALSE
    ]

    # Required sample size for this stratum
    n_h <- as.integer(strata_allocations[stratum])

    # Draw a simple random sample without replacement
```

```r
  sampled_indices <- sample.int(
    n = nrow(stratum_pop),
    size = n_h,
    replace = FALSE
  )

  # Return the selected units
  stratum_pop[sampled_indices, , drop = FALSE]
})

# Combine all strata into one sample
phase1_sample <- do.call(rbind, sample_list)

# Reset row names
rownames(phase1_sample) <- NULL



phase1_sample$N_h <- as.numeric(
  pop_strata_counts[phase1_sample$Strata_ID]
)

phase1_sample$n_ch <- as.numeric(
  strata_allocations[phase1_sample$Strata_ID]
)

# Phase 1 inclusion probability
phase1_sample$Inclusion_Prob_Phase1 <-
  phase1_sample$n_ch / phase1_sample$N_h

# Base design weight
phase1_sample$d_i <-
  1 / phase1_sample$Inclusion_Prob_Phase1



log_odds_resp <- with(
  phase1_sample,

  0.5 +
    1.1 * as.numeric(`Age Group` == "Over 60") +
    0.4 * as.numeric(`Age Group` == "35-59 years") +
    0.8 * as.numeric(`Infrastructure Tier` == "Full Access") -
    0.6 * as.numeric(`Infrastructure Tier` == "Zero Access") -
    1.2 * as.numeric(`Dwelling Type` == "Homeless")
)

# True response probabilities
phase1_sample$True_Theta <-
  1 / (1 + exp(-log_odds_resp))

# Simulate response status:
# 1 = respondent; 0 = non-respondent
```

```r
phase1_sample$Respondent <- rbinom(
  n = nrow(phase1_sample),
  size = 1,
  prob = phase1_sample$True_Theta
)



logit_response_model <- glm(
  Respondent ~
    `Age Group` +
    `Infrastructure Tier` +
    `Dwelling Type`,
  data = phase1_sample,
  family = binomial(link = "logit")
)

# Estimated response probabilities
phase1_sample$Estimated_Theta <- predict(
  logit_response_model,
  newdata = phase1_sample,
  type = "response"
)



phase1_sample$gamma_i <- ifelse(
  phase1_sample$Respondent == 1,
  1 / phase1_sample$Estimated_Theta,
  0
)

# Final sampling weights
phase1_sample$Final_Sampling_Weight <-
  phase1_sample$d_i * phase1_sample$gamma_i



log_odds_vote <- with(
  phase1_sample,

  -0.2 +
    1.3 * as.numeric(`Age Group` == "Over 60") +
    0.6 * as.numeric(`Age Group` == "35-59 years") +
    0.7 * as.numeric(`Infrastructure Tier` == "Full Access") -
    0.9 * as.numeric(`Infrastructure Tier` == "Zero Access") +
    0.5 * as.numeric(`Dwelling Type` == "Traditional Dwelling") -
    1.5 * as.numeric(`Dwelling Type` == "Homeless")
)

# True voting probabilities
phase1_sample$Voting_Probability <-
  1 / (1 + exp(-log_odds_vote))
```

```r
# Generate voting behaviour
phase1_sample$Realized_Behavior <- rbinom(
  n = nrow(phase1_sample),
  size = 1,
  prob = phase1_sample$Voting_Probability
)

# The outcome is observed only for respondents
phase1_sample$Observed_Vote_Outcome <- ifelse(
  phase1_sample$Respondent == 1,
  phase1_sample$Realized_Behavior,
  NA
)


survey_microdata_export <- data.frame(

  Simulation_ID = b,

  Unique_Unit_ID = paste0(
    "SU-",
    sprintf("%05d", phase1_sample$Person_ID)
  ),

  Stratum_Identifier = phase1_sample$Strata_ID,

  Phase1_Inclusion_Prob =
    phase1_sample$Inclusion_Prob_Phase1,

  Estimated_Response_Prob =
    phase1_sample$Estimated_Theta,

  Total_Inclusion_Prob =
    phase1_sample$Inclusion_Prob_Phase1 *
    phase1_sample$Estimated_Theta,

  Design_Weight_di = phase1_sample$d_i,

  Correction_Factor_gamma = phase1_sample$gamma_i,

  Final_Sampling_Weight =
    phase1_sample$Final_Sampling_Weight,

  Survey_Response_Status = ifelse(
    phase1_sample$Respondent == 1,
    "Respondent",
    "Non-Respondent"
  ),

  Vote_Outcome = phase1_sample$Observed_Vote_Outcome,

  Auxiliary_Age_Group = phase1_sample$`Age Group`,
```

```
    Auxiliary_Gender = phase1_sample$Gender,

    Auxiliary_Dwelling_Type = phase1_sample$`Dwelling Type`,

    Auxiliary_Infra_Tier =
      phase1_sample$`Infrastructure Tier`,

    stringsAsFactors = FALSE,
    check.names = FALSE
  )

  # Save this simulation's microdata
  simulation_data[[b]] <- survey_microdata_export


  respondents <- phase1_sample$Respondent == 1

  n_respondents <- sum(respondents)

  response_rate <- mean(respondents)

  # Weighted voting estimate among respondents
  weighted_vote_estimate <- if (n_respondents > 0) {

    respondent_data <- phase1_sample[respondents, ]

    weighted.mean(
      respondent_data$Observed_Vote_Outcome,
      w = respondent_data$Final_Sampling_Weight
    )

  } else {
    NA_real_
  }

  # Save the summary for this simulation
  simulation_summary[[b]] <- data.frame(
    Simulation_ID = b,
    Phase1_Sample_Size = nrow(phase1_sample),
    Number_of_Respondents = n_respondents,
    Number_of_Nonrespondents = sum(!respondents),
    Response_Rate = response_rate,
    Weighted_Vote_Estimate = weighted_vote_estimate
  )

  # Display progress every 50 simulations
  if (b %% 50 == 0) {
    cat("Completed simulation", b, "of", B, "\n")
  }
}
```

```
## Completed simulation 50 of 500
## Completed simulation 100 of 500
## Completed simulation 150 of 500
```

```
## Completed simulation 200 of 500
## Completed simulation 250 of 500
## Completed simulation 300 of 500
## Completed simulation 350 of 500
## Completed simulation 400 of 500
## Completed simulation 450 of 500
## Completed simulation 500 of 500
```

```
# Combine individual-level microdata
all_simulation_data <- do.call(rbind, simulation_data)

rownames(all_simulation_data) <- NULL

# Combine simulation summaries
all_simulation_summary <- do.call(
  rbind,
  simulation_summary
)

rownames(all_simulation_summary) <- NULL



cat("\n=== MONTE CARLO SIMULATION SUMMARY ===\n")
```

```
##
## === MONTE CARLO SIMULATION SUMMARY ===
```

```
cat("Number of simulations:",
    B, "\n")
```

```
## Number of simulations: 500
```

```
cat("Expected sample size per simulation:",
    sum(strata_allocations), "\n")
```

```
## Expected sample size per simulation: 1374
```

```
cat("Total sampled records across simulations:",
    nrow(all_simulation_data), "\n")
```

```
## Total sampled records across simulations: 687000
```

```
cat("Mean number of respondents:",
    mean(all_simulation_summary$Number_of_Respondents), "\n")
```

```
## Mean number of respondents: 1032.024
```

```
cat("Mean response rate:",
    round(mean(all_simulation_summary$Response_Rate), 4), "\n")
```

```
## Mean response rate: 0.7511
```

```
cat("Standard deviation of response rates:",
    round(sd(all_simulation_summary$Response_Rate), 4), "\n")
```

```
## Standard deviation of response rates: 0.0117
```

```
cat("Mean weighted voting estimate:",
    round(mean(all_simulation_summary$Weighted_Vote_Estimate,
```

```
                na.rm = TRUE), 4), "\n")
```

```
## Mean weighted voting estimate: 0.6426
```

```
# Confirm that each simulation has the intended sample size
stopifnot(
  all(all_simulation_summary$Phase1_Sample_Size == 1374)
)



# Export all 500 samples and their associated variables
write.csv(
  all_simulation_data,
  "survey_microdata_500_simulations.csv",
  row.names = FALSE
)

# Export the summary statistics for each simulation
write.csv(
  all_simulation_summary,
  "survey_simulation_summary_500.csv",
  row.names = FALSE
)

cat("\nBoth output files have been created.\n")
```

```
##
## Both output files have been created.
```

Estimating parameters

```
library(survey)

# 1. Specify the simulation settings
B <- 500

# Handle strata containing only one respondent
options(survey.lonely.psu = "adjust")

# 2. Obtain the population stratum sizes
pop_strata_counts <- table(synthetic_population$Strata_ID)

# 3. Create storage for the estimation results
estimation_results <- vector("list", B)


for (b in seq_len(B)) {


  simulation_data_b <- subset(
    all_simulation_data,
    Simulation_ID == b
  )
```

```r
# Retain completed responses only
respondents_data <- subset(
  simulation_data_b,
  Survey_Response_Status == "Respondent" &
    !is.na(Vote_Outcome)
)


respondents_data$N_h <- as.numeric(
  pop_strata_counts[
    as.character(respondents_data$Stratum_Identifier)
  ]
)


# Store an NA result if too few respondents are available
if (nrow(respondents_data) < 2) {

  estimation_results[[b]] <- data.frame(
    Simulation_ID = b,
    Respondent_Count = nrow(respondents_data),
    Point_Estimate = NA_real_,
    Estimated_Variance = NA_real_,
    Standard_Error = NA_real_,
    CI_Lower = NA_real_,
    CI_Upper = NA_real_,
    CV = NA_real_,
    Design_Effect = NA_real_,
    Status = "Insufficient respondents"
  )

  next
}

# Construct the complex survey design
survey_design_b <- svydesign(
  ids = ~1,
  strata = ~Stratum_Identifier,
  weights = ~Final_Sampling_Weight,
  fpc = ~N_h,
  data = respondents_data,
  nest = TRUE
)



vote_estimation_b <- tryCatch(
  svymean(
    ~Vote_Outcome,
    design = survey_design_b,
    deff = TRUE
  ),
  error = function(e) NULL
```

```
)

# Handle any estimation errors
if (is.null(vote_estimation_b)) {

  estimation_results[[b]] <- data.frame(
    Simulation_ID = b,
    Respondent_Count = nrow(respondents_data),
    Point_Estimate = NA_real_,
    Estimated_Variance = NA_real_,
    Standard_Error = NA_real_,
    CI_Lower = NA_real_,
    CI_Upper = NA_real_,
    CV = NA_real_,
    Design_Effect = NA_real_,
    Status = "Estimation failed"
  )

  next
}


# Point estimate
point_estimate <- as.numeric(
  coef(vote_estimation_b)[1]
)

# Estimated variance
estimated_variance <- as.numeric(
  vcov(vote_estimation_b)[1, 1]
)

# Standard error
standard_error <- as.numeric(
  SE(vote_estimation_b)[1]
)

# 95% confidence interval
conf_interval <- tryCatch(
  confint(vote_estimation_b, level = 0.95),
  error = function(e) matrix(
    NA_real_,
    nrow = 1,
    ncol = 2
  )
)

ci_lower <- as.numeric(conf_interval[1, 1])
ci_upper <- as.numeric(conf_interval[1, 2])

# Coefficient of variation
coeff_of_variation <- tryCatch(
  as.numeric(cv(vote_estimation_b)[1]),
```

```
    error = function(e) NA_real_
  )

  # Design effect
  design_effect <- tryCatch(
    as.numeric(deff(vote_estimation_b)[1]),
    error = function(e) NA_real_
  )


  estimation_results[[b]] <- data.frame(
    Simulation_ID = b,
    Respondent_Count = nrow(respondents_data),
    Point_Estimate = point_estimate,
    Estimated_Variance = estimated_variance,
    Standard_Error = standard_error,
    CI_Lower = ci_lower,
    CI_Upper = ci_upper,
    CV = coeff_of_variation,
    Design_Effect = design_effect,
    Status = "Success"
  )

  # Display progress every 50 simulations
  if (b %% 50 == 0) {
    cat("Completed estimation for simulation", b, "of", B, "\n")
  }
}
```

```
## Completed estimation for simulation 50 of 500
## Completed estimation for simulation 100 of 500
## Completed estimation for simulation 150 of 500
## Completed estimation for simulation 200 of 500
## Completed estimation for simulation 250 of 500
## Completed estimation for simulation 300 of 500
## Completed estimation for simulation 350 of 500
## Completed estimation for simulation 400 of 500
## Completed estimation for simulation 450 of 500
## Completed estimation for simulation 500 of 500
```

```
estimation_results_500 <- do.call(
  rbind,
  estimation_results
)

rownames(estimation_results_500) <- NULL


cat("Successful estimations:",
    sum(estimation_results_500$Status == "Success"), "\n")
```

```
## Successful estimations: 500
```

```
cat("Failed or skipped estimations:",
    sum(estimation_results_500$Status != "Success"), "\n")
```

```
## Failed or skipped estimations: 0
```

```
cat("\n--- Point Estimate ---\n")
```

```
##
## --- Point Estimate ---
```

```
print(summary(estimation_results_500$Point_Estimate))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6015  0.6324  0.6428  0.6426  0.6524  0.6823
```

```
cat("\n--- Estimated Variance ---\n")
```

```
##
## --- Estimated Variance ---
```

```
print(summary(estimation_results_500$Estimated_Variance))
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
## 0.0001647 0.0001853 0.0001891 0.0001894 0.0001934 0.0002046
```

```
cat("\n--- Standard Error ---\n")
```

```
##
## --- Standard Error ---
```

```
print(summary(estimation_results_500$Standard_Error))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.01284 0.01361 0.01375 0.01376 0.01391 0.01430
```

```
cat("\n--- Lower 95% Confidence Limit ---\n")
```

```
##
## --- Lower 95% Confidence Limit ---
```

```
print(summary(estimation_results_500$CI_Lower))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.5736  0.6053  0.6157  0.6156  0.6255  0.6563
```

```
cat("\n--- Upper 95% Confidence Limit ---\n")
```

```
##
## --- Upper 95% Confidence Limit ---
```

```
print(summary(estimation_results_500$CI_Upper))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.6295  0.6595  0.6698  0.6695  0.6794  0.7083
```

```
cat("\n--- Coefficient of Variation ---\n")
```

```
##
## --- Coefficient of Variation ---
```

```
print(summary(estimation_results_500$CV))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 0.01904 0.02100 0.02141 0.02143 0.02184 0.02367
```

```
cat("\n--- Design Effect ---\n")
```

```
##
## --- Design Effect ---
```

```
print(summary(estimation_results_500$Design_Effect))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.8691  0.9324  0.9490  0.9488  0.9658  1.0187
```

```
valid_estimates <- estimation_results_500$Point_Estimate
valid_estimates <- valid_estimates[is.finite(valid_estimates)]

cat("\n--- Monte Carlo Variability ---\n")
```

```
##
## --- Monte Carlo Variability ---
```

```
cat("Mean point estimate:",
    mean(valid_estimates), "\n")
```

```
## Mean point estimate: 0.6425721
```

```
cat("Empirical variance of estimates:",
    var(valid_estimates), "\n")
```

```
## Empirical variance of estimates: 0.0002079629
```

```
cat("Monte Carlo standard deviation:",
    sd(valid_estimates), "\n")
```

```
## Monte Carlo standard deviation: 0.01442092
```

```
cat("Mean estimated variance:",
    mean(estimation_results_500$Estimated_Variance,
         na.rm = TRUE), "\n")
```

```
## Mean estimated variance: 0.0001893812
```

```
cat("Mean estimated standard error:",
    mean(estimation_results_500$Standard_Error,
         na.rm = TRUE), "\n")
```

```
## Mean estimated standard error: 0.01375985
```

```
write.csv(
  estimation_results_500,
  "Section_D_estimation_results_500.csv",
  row.names = FALSE
)

cat("\nResults saved to Section_D_estimation_results_500.csv\n")
```

```
##
## Results saved to Section_D_estimation_results_500.csv
```

##Section E:Bias and estimation of parameters,variance etc

```
# True population voting proportion
true_proportion <- 0.6730
```

```r
calculate_performance <- function(results, true_proportion) {

  # Retain repetitions with complete estimates
  valid <- complete.cases(
    results[, c(
      "Point_Estimate",
      "Standard_Error",
      "CI_Lower",
      "CI_Upper"
    )]
  )

  results <- results[valid, ]

  # Extract estimates and confidence intervals
  estimates <- results$Point_Estimate
  estimated_se <- results$Standard_Error
  lower <- results$CI_Lower
  upper <- results$CI_Upper

  # Number of valid simulations
  B <- length(estimates)

  if (B == 0) {
    stop("No valid simulation results are available.")
  }

  # Estimation errors
  errors <- estimates - true_proportion

  # 1. Empirical bias
  empirical_bias <- mean(errors)

  # 2. Relative bias (%)
  relative_bias <- if (true_proportion != 0) {
    100 * empirical_bias / true_proportion
  } else {
    NA_real_
  }

  # 3. Empirical standard error
  empirical_se <- sd(estimates)

  # 4. Average estimated standard error
  average_estimated_se <- mean(estimated_se)

  # 5. Mean squared error
  mse <- mean(errors^2)

  # 6. Root mean squared error
  rmse <- sqrt(mse)

  # 7. 95% confidence interval coverage
```

```r
  covered <- lower <= true_proportion &
             upper >= true_proportion

  coverage <- mean(covered)

  # 8. Monte Carlo standard error of coverage
  mc_se_coverage <- sqrt(
    coverage * (1 - coverage) / B
  )

  # 9. Average confidence interval width
  average_ci_width <- mean(upper - lower)

  # Return results
  data.frame(
    Valid_Repetitions = B,
    True_Proportion = true_proportion,
    Empirical_Bias = empirical_bias,
    Relative_Bias_Percent = relative_bias,
    Empirical_SE = empirical_se,
    Average_Estimated_SE = average_estimated_se,
    MSE = mse,
    RMSE = rmse,
    Coverage = coverage,
    Coverage_Percent = 100 * coverage,
    MC_SE_Coverage = mc_se_coverage,
    MC_SE_Coverage_Percentage_Points =
      100 * mc_se_coverage,
    Average_CI_Width = average_ci_width
  )
}
selected_design_results <- calculate_performance(
  results = estimation_results_500,
  true_proportion = true_proportion
)

selected_design_results$Design <- "Selected Design"
selected_design_results
```

```
##   Valid_Repetitions True_Proportion Empirical_Bias Relative_Bias_Percent
## 1               500           0.673    -0.03042793             -4.521238
##   Empirical_SE Average_Estimated_SE         MSE      RMSE Coverage
## 1   0.01442092           0.01375985 0.001133406 0.0336661    0.418
##   Coverage_Percent MC_SE_Coverage MC_SE_Coverage_Percentage_Points
## 1             41.8     0.02205792                          2.205792
##   Average_CI_Width          Design
## 1       0.05393763 Selected Design
```

```r
# Print MC SE of Coverage for both designs
cat("SRSWOR MC SE of Coverage:", selected_design_results$MC_SE_Coverage_Percentage_Points, "percentage p
```

```
## SRSWOR MC SE of Coverage: 2.205792 percentage points
```

Section B

##Plots:

```
N <- 10000          # Population size
z <- 1.96           # 95% Confidence Level Z-score
P_baseline <- 0.50  # Baseline proportion parameter value

# Assumed parameter values for Question 7 (25%, 50%, 75%, 100% of baseline)
percentages <- c(0.25, 0.50, 0.75, 1.00)
P_values <- percentages * P_baseline

# Define a range of absolute margins of error (E) for the X-axis (from 0.01 to 0.10)
E_range <- seq(0.01, 0.10, length.out = 500)

# --- Function to Calculate Sample Size with FPC ---
calculate_sample_size <- function(P, E, N, z = 1.96) {
 # 1. Initial sample size (Cochran's formula)
 n0 <- (z^2 * P * (1 - P)) / (E^2)
 # 2. Apply Finite Population Correction
 n <- n0 / (1 + (n0-1 / N))
 return(n)
}

# --- Plotting using Base R (No external packages required) ---

# Set up colors for the 4 curves
colors <- c("green", "red", "orange", "blue")

# Initialize an empty plot with proper dimensions and labels
plot(NULL, xlim = c(0.01, 0.10), ylim = c(0, 2000),
    xlab = "Absolute Margin of Error (E) [CI Half-Width]",
    ylab = "Required Minimum Sample Size (n)",
    main = "Sensitivity Analysis: Sample Size vs. Margin of Error",
    font.main = 2, cex.main = 1.2)

grid(nx = NULL, ny = NULL, lty = 2, col = "gray80")

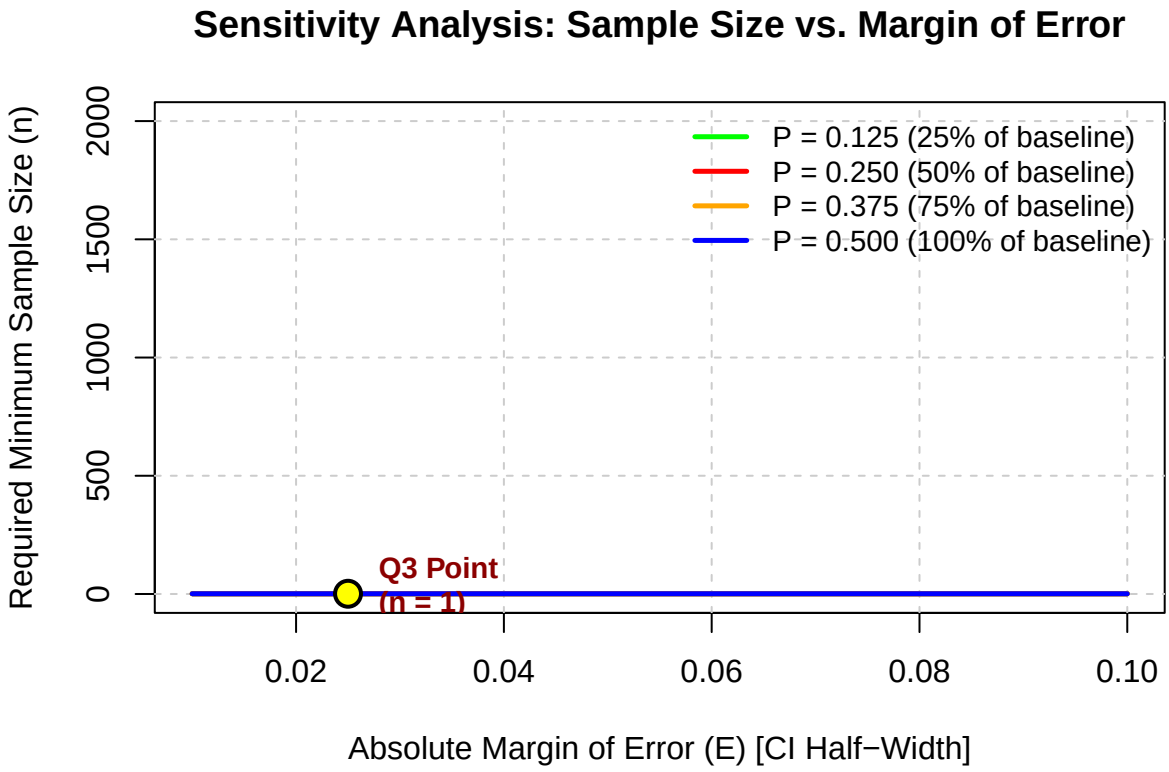
# Loop through each P value and add its curve to the plot
for (i in 1:length(P_values)) {
 P <- P_values[i]
 n_values <- calculate_sample_size(P, E_range, N, z)
 lines(E_range, n_values, col = colors[i], lwd = 2.5)
}


E_q3 <- 0.05 * P_baseline
n_q3 <- calculate_sample_size(P_baseline, E_q3, N, z)


points(E_q3, n_q3, col = "black", bg = "yellow", pch = 21, cex = 1.8, lwd = 2)


text(E_q3, n_q3, labels = paste0("Q3 Point\n(n = ", ceiling(n_q3), ")"),
    pos = 4, offset = 0.8, col = "darkred", font = 2, cex = 0.9)
```

```r
# Add the legend to distinguish the lines
legend_labels <- sprintf("P = %.3f (%d%% of baseline)", P_values, percentages * 100)
legend("topright", legend = legend_labels, col = colors, lty = 1, lwd = 2.5, bty = "n", cex = 0.9)
```



##Variance sensitivity analysis:

```r
# Strata population sizes (N_h) from your synthetic data
N_h <- c(1376, 2422, 179, 2609, 1644, 92, 299, 1342, 37)
N <- sum(N_h)           # Total population (10,000)
n <- 1333               # Your sample size
f <- n / N              # Sampling fraction (0.1333)
W_h <- N_h / N          # Strata weights

# Investigate P from 0.10 to 0.90
P_range <- seq(0.10, 0.90, by = 0.01)

# Function to calculate V(p_st) using formula 5.59 assuming P_h = P across strata
calculate_stratified_variance <- function(P, W_h, n, f) {
 # For a conservative sensitivity analysis, assume P_h = P within strata
 sum_W_P_Q <- sum(W_h * P * (1 - P))
 V_p_st <- ((1 - f) / n) * sum_W_P_Q
 return(V_p_st)
}

# Calculate variance for each P
variances <- sapply(P_range, calculate_stratified_variance, W_h = W_h, n = n, f = f)

# Plot the Variance Curve
plot(P_range, variances, type = "l", lwd = 2.5, col = "darkblue",
    xlab = "Assumed Proportion (P)", ylab = "Stratified Variance V(p_st)",
```

```
    main = "Question 9: Stratified Variance vs. Assumed P")
grid(lty = 2, col = "gray80")

# Highlight the maximum at P = 0.50
abline(v = 0.50, col = "red", lty = 3, lwd = 2)
points(0.50, max(variances), col = "red", pch = 19, cex = 1.5)
text(0.50, max(variances), labels = paste0("Max Variance\n(P = 0.50)"),
    pos = 3, col = "red", font = 2, cex = 0.9)
```

